ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 3

Sumário

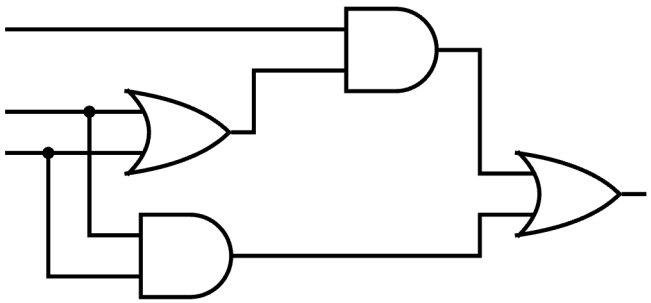
- Implementação de funções com circuitos digitais
- Introdução ao VHDL
- Exercícios de simplificação de funções com expressões algébricas

Implementação de funções com circuitos digitais

- Os circuitos digitais têm entradas e saídas
  - As variáveis da função lógica são as entradas do circuito
  - A função lógica é a saída do circuito

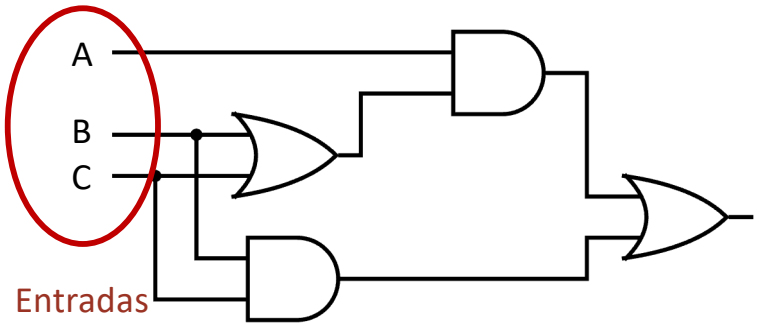
Implementação de funções com circuitos digitais

- Os circuitos digitais têm entradas e saídas
  - As variáveis da função lógica são as entradas do circuito
  - A função lógica é a saída do circuito



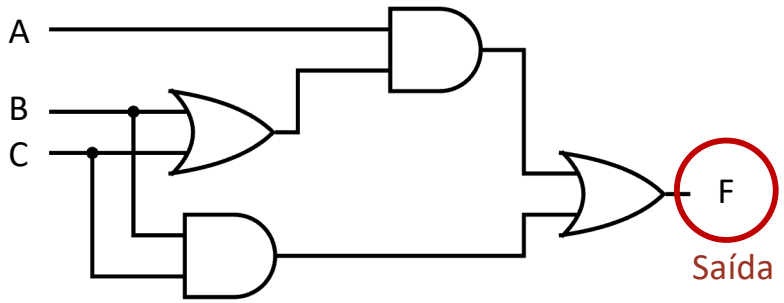
Implementação de funções com circuitos digitais

- Os circuitos digitais têm entradas e saídas
  - As variáveis da função lógica são as entradas do circuito
  - A função lógica é a saída do circuito



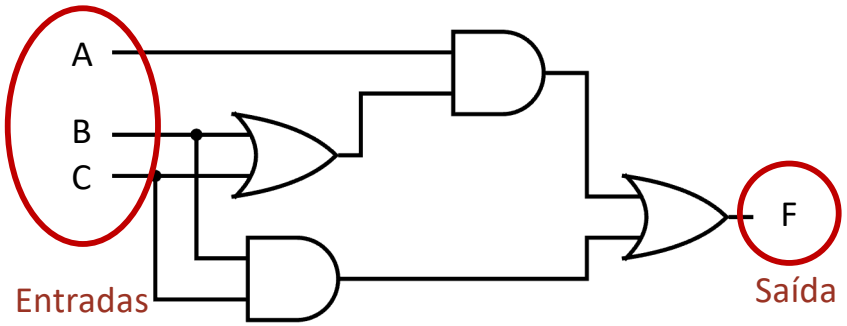
Implementação de funções com circuitos digitais

- Os circuitos digitais têm entradas e saídas
  - As variáveis da função lógica são as entradas do circuito
  - A função lógica é a saída do circuito



Implementação de funções com circuitos digitais

- Os circuitos digitais têm entradas e saídas
  - As variáveis da função lógica são as entradas do circuito
  - A função lógica é a saída do circuito



$$F(A, B, C) = BC + A(B + C)$$

Implementação de funções com circuitos digitais

- A implementação das funções lógicas pode ser realizada usando diversas tecnologias
  - Exemplos:
    - Circuitos integrados discretos (como os usados no Laboratório 1)
    - Lógica programável (FPGAs, PLAs, PALs…)
    - Circuitos integrados dedicados (ASICs)
- Para circuitos de maior complexidade, usam-se linguagens de descrição de hardware (HDL) para descrever o circuito a implementar, p. ex.:
  - VHDL
  - Verilog
- Utilizando ferramentas EDA, é possível gerar a partir da descrição:
  - Ficheiros de programação – no caso de se usar lógica programável
  - Máscaras de fabrico do circuito integrado – no caso de um circuito dedicado

EDA – Electronic Design Automation HDL – Hardware Description Language

Implementação de funções com circuitos integrados discretos

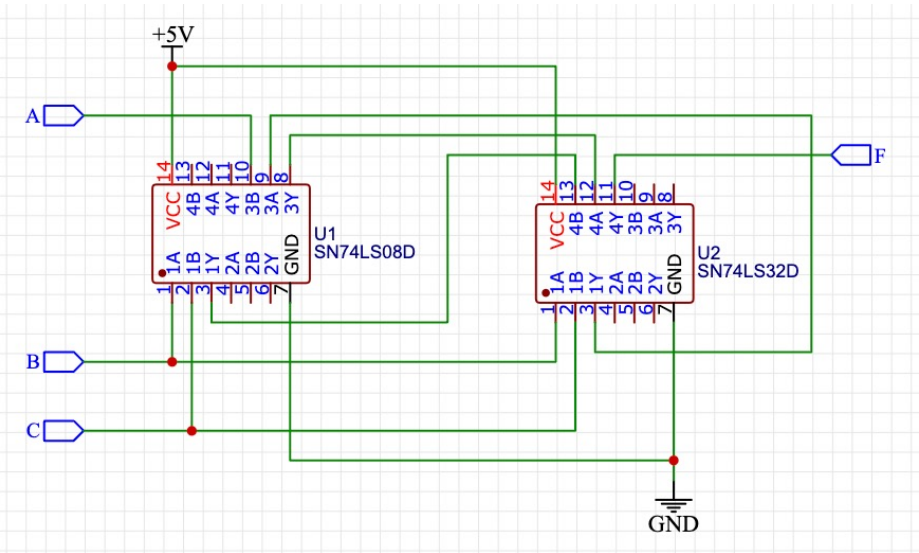
- Os circuitos digitais utilizam grandezas físicas (e.g. tensão, corrente) para representar (codificar) os valores lógicos binários da álgebra de Boole
  - Cada tecnologia tem o seu próprio mapeamento

- No caso da família lógica 74LSxxx, os valores lógicos binários são representados em dois intervalos de tensão
  - Valor de tensão HIGH – [2; 5] V
  - Valor de tensão LOW – [0; 0,8] V

- Adicionalmente, é necessário mapear os valores lógicos numéricos 1 e 0 nos possíveis valores da respetiva grandeza física
  - No caso da família lógica 74LSxxx, o valor lógico 1 pode ser mapeado no:
    - Valor de tensão HIGH – caso em que se denomina a entrada/saída como *active_high*
    - Valor de tensão LOW – caso em que se denomina a entrada/saída como *active_low*
  - Um mesmo circuito pode ter algumas das suas entradas/saída *active_high* e outras *active_low*
    - É necessário verificar as características do respetivo circuito

Esquema elétrico com circuitos integrados discretos

- Utilizando a ferramenta EasyEDA Standard é possível desenhar o esquema elétrico do circuito a ser implementado



Implementação de funções com circuitos digitais discretos

- Resumo para o laboratório 1:
  - vamos apenas utilizar circuitos digitais com **entradas e saídas *active_high***

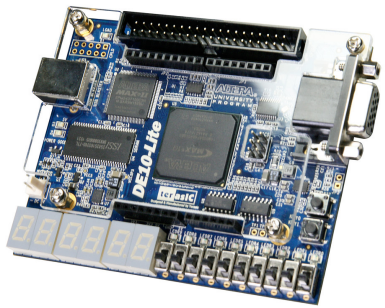
Valor lógico (número)	Valor lógico (tensão)	Valor de tensão
1	HIGH	[2; 5] V
0	LOW	[0; 0,8] V

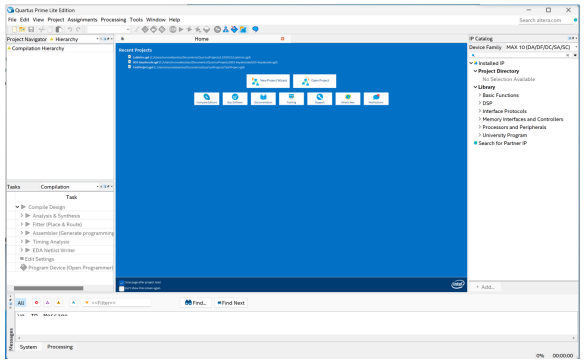
Mapeamento entre a representação numérica do valor lógico e o correspondente valor de tensão nos circuitos digitais 74LS00, 74LS02 e 74LS04 (entradas e saídas *active_high*)

Implementação de funções com lógica programável - FPGA

- Placa de desenvolvimento a ser usada no laboratório:
  - MAX DE10-Lite
    - Contém uma FPGA Altera MAX 10
- Ferramentas EDA
  - Quartus Prime Lite Edition
- Linguagem de descrição de hardware
  - VHSIC Hardware Description Language (VHDL)
    - VHSIC – Very High Speed Integrated Circuit

placa que vamos usar





Descrição do circuito em VHDL

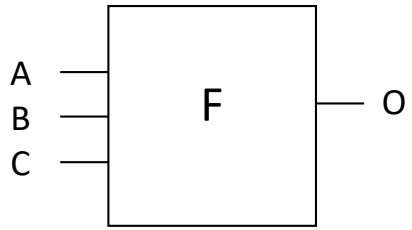
- Estrutura de um ficheiro VHDL
  - *Entity* (entidade)
  - *Architecture* (arquitetura)

Descrição do circuito em VHDL

- Estrutura de um ficheiro VHDL
  - *Entity* (entidade)

    da uma visao do que é o circuito em termos de entradas e saidas

    - Utilizada para descrever o interface (entradas, saídas e configurações) do circuito
    - Corresponde à vista do exterior do circuito



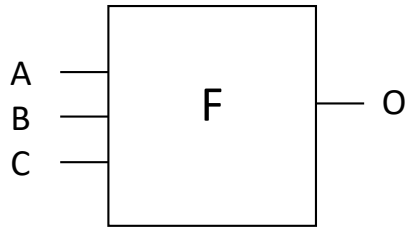
$$O = F(A, B, C) = BC + A(B + C)$$

  - *Architecture* (arquitetura)

como é que ºe o comportamento do sistema.
forma estrutural - relaçao entre componentes
comportamental -

Descrição do circuito em VHDL

- Estrutura de um ficheiro VHDL
  - *Entity* (entidade)
    - Utilizada para descrever o interface (entradas, saídas e configurações) do circuito
    - Corresponde à vista do exterior do circuito



$$O = F(A, B, C) = BC + A(B + C)$$

  - *Architecture* (arquitetura)
    - Utilizada para descrever a funcionalidade da entidade
    - Pode ser descrita usando duas formas:
      - *Structural* (estrutural) – descrição usando outros componentes
      - *Behavioral* (comportamental) – descrição do comportamento

vamos dar prioridade e so vamos usar as estruturais nesta cadeira.

Descrição do circuito em VHDL

um signal equivale a um fio

- *Signals* (sinais)
  - Um *signal* é um objeto que representa um histórico de valores de um determinado tipo de dados
  - É utilizado para representar ligações entre entradas, saídas e outros sinais
  - Um *signal* está associado a um tipo de dados, por exemplo:
    - tipo ***bit***, que representa os dois valores possíveis da lógica de boole: '0' e '1'
    - tipo ***std_logic***, que extende os valores usados na lógica de boole para outros que ajudam a caracterizar o funcionamento do circuito elétrico:
      - '0' e '1'
      - 'Z' – alta impedância
      - '-' – indiferente (*don't care*)
      - 'U' – indefinido (undefined)
      - 'X' – indeterminado (unkown)

por exempli para o indeterminado, conseguimos saber que nao tá ligado a nada vs

  - Exemplo da sintaxe de definição
    - signal enable : std_logic;

Descrição do circuito em VHDL

- A descrição de um barramento (*bus*) recorre à utilização de um vetor de ***n*** elementos, o qual é equivalente à utilização de ***n*** elementos separados
- Exemplo de definição
  - usando ***3*** elementos separados:
    - signal DIN0, DIN1, DIN2 : std_logic;
  - usando um vetor de ***3*** elementos:
    - signal DIN : std_logic_vector (2 downto 0);

vou usar um vector em vez destes tres sinais diferentes

- Os elementos de um vetor podem ser acedidos com a seguinte sintaxe:
  - DIN(0), DIN(1), DIN(2)

Descrição do circuito em VHDL

- Operadores lógicos
  - not
  - and
  - or
  - nand
  - nor
  - xor
  - xnor
- A precedência entre os operadores lógicos é definida por cada ferramenta, com excepção do operador *not*, que tem sempre precedência sobre os restantes
- Melhor abordagem é a utilização de parentesis para definir a ordem das operações
  - Exemplo: (a and b) or (b and not c)

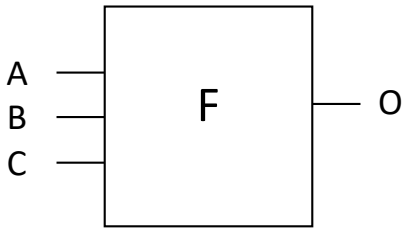
Descrição do circuito em VHDL

- Estrutura de um ficheiro VHDL

```
library ieee;
use ieee.std_logic_1164.all;

entity ? is
   port (



   );
end ?;

architecture ?????????? of ? is

begin


end ??????????;
```

Descrição do circuito em VHDL

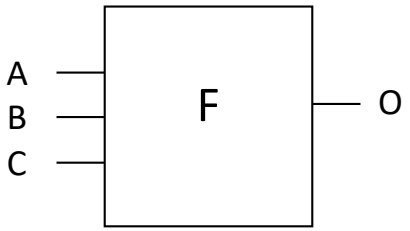
- Estrutura de um ficheiro VHDL




```
library ieee;
use ieee.std_logic_1164.all;

entity F is
   port (



   );
end F;

architecture ?????????? of F is

begin


end ??????????;
```

Descrição do circuito em VHDL

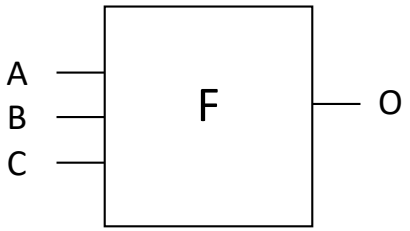
- Estrutura de um ficheiro VHDL




```
library ieee;
use ieee.std_logic_1164.all;

entity F is
   port (
      A : in std_logic;
      B : in std_logic;
      C : in std_logic;
      O : out std_logic
   );
end F;


architecture ????????? of F is

begin


end ??????????;
```

Descrição do circuito em VHDL

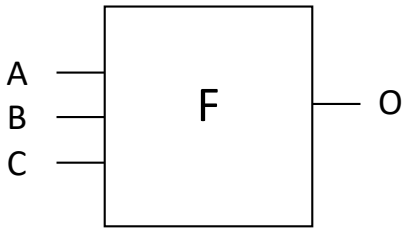
- Estrutura de um ficheiro VHDL




```
library ieee;
use ieee.std_logic_1164.all;

entity F is
   port (
      A : in std_logic;
      B : in std_logic;
      C : in std_logic;
      O : out std_logic
   );
end F;

architecture behavioral of F is

begin


end behavioral;
```

Descrição do circuito em VHDL

- Estrutura de um ficheiro VHDL



$$O = F(A, B, C) = BC + A(B + C)$$

```
library ieee;
use ieee.std_logic_1164.all;

entity F is
   port (
      A : in std_logic;
      B : in std_logic;
      C : in std_logic;
      O : out std_logic
   );
end F;

architecture behavioral of F is

begin

   O <= (B and C) or (A and (B or C));

end behavioral;
```

isto é um array

Temas abordados

Sumário

- Implementação de funções com circuitos digitais
- Introdução ao VHDL
- Exercícios de simplificação de funções com expressões algébricas

Próxima aula

- Exercícios de simplificação de funções com mapa de Karnaugh